



FeatureExtractor

```
resnet50 = models.resnet50(pretrained=True)
self.features = nn.Sequential(
    resnet50.conv1, resnet50.bn1, nn.ReLU(inplace=False), resnet50.maxpool
)
```

用 ResNet50 的 stem (7×7 conv, bn, relu, maxpool) , 输出通道 64, 空间尺寸变为 H/4×W/4

GradientVarianceFocusHead

核心思想：清晰区域边缘强、局部方差大。对每张图的特征图做：

1. 用 Sobel 算子求梯度强度：grad = mean(|Gx|+|Gy|, dim=1) 得到 [B,1,h,w]
2. 计算局部均值后求方差：var = mean((x - avgpool(x))^2, dim=1) 得到 [B,1,h,w]
3. 两者各做 sigmoid 归一，拼接成2通道，再过一个小卷积头产生 1通道 logits。

下面把 GradientVarianceFocusHead 逐行拆开讲清楚：它在干什么、为什么这么设计、数学意义、张量尺寸、可微性与数值细节，以及可以改进的点。

这个头的总体作用

给**一张特征图**（来自 backbone 的 [B,C,H,W] ）做两类**清晰度统计**：

1. **梯度强度**（边缘/细节程度）
2. **局部方差**（纹理/细节起伏）

把这两张统计图拼起来作为 2 通道“证据”，再用一个很轻的卷积小头把证据映射成**单通道的 logits**（注意：是 *logits*，不是概率/掩码）。后面会把两幅输入图的 logits 对齐到同分辨率，再做 softmax 得到每像素属于“选图1/选图2”的概率。

代码逐段解释

初始化部分

```
self.alpha = nn.Parameter(torch.tensor(0.5))
```

- 当前实现里**没有使用到** `self.alpha`。它通常用于做可学习的权重（例如后面把 grad 与 var 以 `alpha` 融合），但现在没用到，等同于冗余参数。

```
self.conv = nn.Sequential(  
    nn.Conv2d(2, 8, 3, padding=1),  
    nn.ReLU(inplace=True),  
    nn.Conv2d(8, 1, 3, padding=1),  
)
```

- 输入是 2 通道（梯度强度、方差），输出 1 通道 logits。
- 两个 3×3 卷积 + 中间 ReLU，属于**极轻量**的非线性映射，用来学习“怎样把这两类证据组合成一张决策图”。

```
self.register_buffer('sobel_x', tensor(...).view(1,1,3,3))  
self.register_buffer('sobel_y', tensor(...).view(1,1,3,3))
```

- 注册为 **buffer**（不是可训练参数），表示固定的 Sobel 核（x/y 方向梯度）。
- `view(1,1,3,3)` 形状上等同单通道卷积核，后面会为每个输入通道复制一份并用 `groups=C` 做**深度可分离卷积**来逐通道求梯度。

forward 计算

```
grad_x = F.conv2d(  
    x,  
    self.sobel_x.repeat(x.size(1),1,1,1), # 变成 [C,1,3,3]  
    groups=x.size(1), # 深度卷积：每个通道各自与一个 [1,1,3,3] 核卷  
    padding=1  
)  
grad_y = F.conv2d( ... 同理 ... )
```

- 输入 `x: [B, C, H, W]`
- 通过 `groups=C`，卷积会把每个通道独立地与对应的 Sobel 核相乘求梯度。
- `padding=1` 确保输出空间尺寸仍是 `[H,W]`。
- 得到 `grad_x, grad_y` 形状仍是 `[B, C, H, W]`。

```
grad = (grad_x.abs() + grad_y.abs()).mean(dim=1, keepdim=True) # [B,1,H,W]
```

- 这里用 **L1 梯度幅值近似**：`|Gx| + |Gy|`。
- 然后在 **通道维做均值** (`mean(dim=1)`)，得到单通道的**整体梯度强度图**。
- 用通道均值而不是最大/加权，是一种**简单稳定**的聚合，让不同语义/尺度的特征通道贡献平均化。

```
x_mean = F.avg_pool2d(x, 3, stride=1, padding=1) # 局部均值
var = ((x - x_mean)**2).mean(dim=1, keepdim=True) # [B,1,H,W]
```

- 用 3×3、步长 1 的平均池化得到每个通道的**局部均值**，再计算 `(x - x_mean)^2` 并在通道维取均值，得到单通道**局部方差图**。
- **直觉**：清晰区域纹理起伏大、局部方差更高。

```
grad_n = torch.sigmoid(grad)
var_n = torch.sigmoid(var)
```

- 把两个统计做 **sigmoid** 归一到 (0,1)（不是把它们当最终概率，只是把动态范围“捏”到稳定的区间，方便后续小头学习）。
- 如果不做这步，`grad/var` 的数值范围可能相差很大，卷积小头更难学到合理权重。

```
stats = torch.cat([grad_n, var_n], dim=1) # [B,2,H,W]
logits = self.conv(stats) # [B,1,H,W]
return logits
```

- 拼接两种证据，交给小头 `conv(2→8→1)` 去**学习组合关系**（比如：某些区域梯度高但方差低、或相反时该怎么判）。
- 输出是 **logits**（任意实数），后续再与另一张图的 logits 拼在一起，经 softmax 得到真正的**权重/概率**。

数学直观（简化表述）

- 设特征图为 $(x \in \mathbb{R}^{C \times H \times W})$ 。

梯度强度图（单通道）

$$G(i, j) = \frac{1}{C} \sum_{c=1}^C (|(x_c * S_x)(i, j)| + |(x_c * S_y)(i, j)|)$$

其中 (S_x, S_y) 是 Sobel 核, $(*)$ 为卷积。

局部方差图（单通道）

$$\mu_c(i, j) = \text{avgpool} * 3 \times 3(x_c)(i, j),$$

$$V(i, j) = \frac{1}{C} \sum_{c=1}^C (x_c(i, j) - \mu_c(i, j))^2$$

归一并拼接

$$\hat{G} = \sigma(G), \quad \hat{V} = \sigma(V), \quad S = \text{concat}[\hat{G}, \hat{V}]$$

小头映射成 logits

$$\text{logits} = \phi(S) \in \mathbb{R}^{1 \times H \times W}$$

(ϕ) 是两层 3×3 Conv + ReLU 的轻量 CNN。

为什么输出 logits 而不是直接 mask?

- **数值更稳**：二分类/二选一问题里，先在 logits 空间做线性/加性融合（比如再加上 CLIP 先验的 log 概率），最后再 softmax 成概率，**比直接在概率上做算术更稳健**。
 - **易于融入先验**：你主干里把 CLIP 的先验概率取 `log` 后作为 **logits 偏置** 加进去，这在 logits 空间是自然操作。
 - **可配合任意监督**：针对 logits 用标准的交叉熵/ Dice loss 等都很自然。
-

可微性与训练

- 除 Sobel 核 (buffer, 固定不训练) 以外, 所有步骤 (conv2d、avg_pool2d、拼接、conv 小头) 都是**可微**的, 梯度能回传给 conv 小头以及前面的 backbone (如果没冻结)。
 - sigmoid 只是一种数值压缩, 它的梯度也可正常回传。
-

尺寸与复杂度

- 输入 [B,C,H,W], 输出 [B,1,H,W]。
 - 复杂度基本是 **两次 depthwise 3×3 Sobel 卷积 + 一次 3×3 平均池化 + 两个 3×3 标准卷积**, 相对 backbone 来说很轻。
-

数值/实现细节的合理性

- groups=C + repeat(C,1,3,3) 是**标准做法**, 表示每个通道用自己的 Sobel 卷积核 (其实是同一核的复制)。
 - padding=1 保持分辨率不变, 避免边界缩小。
 - 对 grad/var 做 sigmoid 是**归一化**而非“当概率”, 别被名字误导。
 - inplace=True 的 ReLU 在这里一般**没问题**, 因为作用在 self.conv 内部新生成的张量上, 不会破坏外部计算图。
-

可以考虑的改进

1. 用上 alpha :

例如在卷积头前做一层可学加权:

$$S' = \text{concat}[\sigma(G), \alpha \cdot \sigma(V)]$$

或者把 alpha 放进 conv 的第一层权重初始化里。

2. 标准化替代 sigmoid:

对 $(\hat{G}, \hat{V})$ 使用 BatchNorm2d / LayerNorm 或者 min-max 标准化, 可能比 sigmoid 更线性、梯度更友好 (看数据分布而定)。

3. 多尺度统计：

不仅 3×3 的均值/方差，也可以加 5×5 、 7×7 的并行通道再交给小头融合，捕获不同尺度的清晰度线索。

4. 更强的通道聚合：

现在对 C 维直接 `mean`。可以尝试 1×1 卷积先把通道压到几维（可学习聚合），再做统计，或者直接在特征域上学一个“清晰度通道投影”。

这个模块把**清晰区域边缘更强、纹理起伏更大**这两个经典直觉，变成了**可微的、可学习组合的单通道 logits 估计器**，为后续的两图对比与 softmax 融合提供了**稳定、物理可解释**的证据。虽然有综述文献说了用 梯度强度 + 方差作为“焦点测度”的学术依据，大师没有直接借鉴别人的东西，具体的实现和别人的不同的，更多是把经典焦点测度（gradient magnitude, local variance）嵌入 CNN 可微头中，还是挺多自己提出的东西，然后也有还可以改进的内容，可以后续加上。

CLIP使用模块

1) CLIP 的角色定位

把 CLIP 当作**“清晰/模糊先验评估器”**：

- 用文本提示（prompts）分别刻画“清晰”（clear）和“模糊”（unclear）的语义原型；
- 让 CLIP 对**局部块（patch）**进行图文匹配，得到每个块是“清晰”还是“模糊”的相对支持度；
- 再把块级分数“铺回去”形成**像素级软焦点图（two-channel）**，作为**先验概率**喂给主模型，与可微的 Gradient/Variance 头输出的 **logits** 在 logits 域融合，最后 softmax 成像素权重用于融合。

CLIP 给出“哪儿更像清晰/模糊”的软分布，主模型再结合可学习证据做“后验决策”。

2) CLIPFocusClassifier: 如何把文本与图像“对齐”

2.1 文本侧：清晰/模糊提示 → 文本特征（一次性计算+缓存）

```
self.clear_prompts = [
    "a clear and sharp image", "a focused photograph", ...
]
self.unclear_prompts = [
    "a blurry and unclear image", "an unfocused photograph", ...
]
self.clear_text_features = self._encode_text_prompts(self.clear_prompts)
self.unclear_text_features = self._encode_text_prompts(self.unclear_prompts)
```

- 用 `clip.tokenize(prompts)` → `clip_model.encode_text()` 得到每条文本的向量，然后 **L2 归一化**：

$$t_i \leftarrow \frac{t_i}{|t_i|_2}$$

- 做了**多提示最大化**（见后面“图像相似度”）来增强鲁棒性：对一组描述里挑相似度最大的那条。

2.2 图像侧：补丁（patch）化 + CLIP 统计的归一化

考虑了**两套均值/方差**：

- 训练/数据管线常见的 ImageNet 统计（`dataset_mean/std`）；
- CLIP 官方需配的统计（`clip_mean/std`）。

在 `_compute_focus_scores` 中先把输入 patch **反归一**回 [0,1]，然后再按 **CLIP mean/std** 标准化，确保输入分布**契合 CLIP 预期**：

```
patches = patches * dataset_std + dataset_mean # 回到[0,1]
patches = (patches - clip_mean) / clip_std      # 转CLIP分布
```

随后把每个 patch 插值为 **224×224**（ViT-B/32 的默认输入尺寸）再喂入

`clip_model.encode_image`，并 **L2 归一化**：

$$f \leftarrow \frac{f}{\|f\|_2}$$

2.3 图文相似度 → “清晰/模糊”两个簇的对比

对每个 patch 的图像特征 (f):

- 与清晰提示簇 ($t^{(c)} * k$) 做点积 (就是 cosine, 相当于 CLIP 相似度), 取**最大值**:

$$s * \text{clear} = \max_k f^\top t_k^{(c)}$$

- 与模糊提示簇 ($t^{(u)} * k$) 同理:

$$s * \text{unclear} = \max_k f^\top t_k^{(u)}$$

- 计算**归一化的清晰得分** (你代码里的 `focus_score`) :

$$\text{focus} = \frac{s_{\text{clear}}}{s_{\text{clear}} + s_{\text{unclear}} + \varepsilon} \in (0, 1)$$

- 同时记录一个**置信度** (`confidence = max(s_clear, s_unclear)`), 可度量该块“像两类原型之一”的强度。

这是**二元对比归一**, 不是 softmax, 但含义等价于“清晰相对于模糊的支持度比例”。

3) get_pixel_focus_map: 从块级到像素级的软图

使用 `unfold/fold` 机制:

- `unfold(kernel=patch_size, stride=stride)` 把整图 `[C,H,W]` 切成 L 个 patch (通常重叠)。
- 对每个 patch 计算 `focus_score` (如上)。得到长度 L 的一维分数列。
- 把每个分数**均匀铺**回对应 patch 的像素 (复制 `patch_size2` 次), 再用 `fold` 叠回到 `[H,W]`:
 - 同一个像素可能被多个重叠 patch 覆盖, 用**计数图** `count_map` 做了平均 (除法), 相当于一种**滑动窗口平滑**。
- 输出的是 **单通道清晰度热力图** (值越大越清晰)。

这一步是把**块级语义判断**变为**像素级软先验**，对噪声鲁棒（重叠平均），也能自然平滑边界。

4) CLIPPixelFusion: 两幅图的先验 → 两通道概率

对 `img1`、`img2` 分别生成各自的像素清晰度图 `focus_map1` / `focus_map2`，再做：

```
focus_maps = torch.cat([focus_map1, focus_map2], dim=1) # [B,2,H,W]
focus_maps = torch.clamp(focus_maps, min=1e-6)
focus_maps = focus_maps / (focus_maps.sum(dim=1, keepdim=True) + 1e-6)
```

- 得到**两通道先验概率**，逐像素相加为 1。
- 这是“CLIP 判断图像1/图像2谁更清晰”的**软概率**，可视为 $(p_{\text{CLIP}}(y = i | x))$ 。

5) 与可微 logits 融合（主模型内的关键数学）

在 `MultiFocusFusionModel.forward`：

1. **可微证据**（来自 FeatureExtractor+GradientVarianceFocusHead）得到两个上采样后的 **logits**：

$$\ell_1(x), \ell_2(x) \Rightarrow L_{\text{base}} = [\ell_1, \ell_2]$$

2. CLIP 先验是概率分布 `[B,2,H,W]`，取对数 → **logits 偏置**：

$$L_{\text{prior}} = \log p_{\text{CLIP}}$$

3. 线性门控融合（你用 `clip_weight=λ`）：

$$L_{\text{comb}} = (1 - \lambda), L_{\text{base}} + \lambda, L_{\text{prior}}$$

直觉：(λ) 大时更信 CLIP，小时更信可微头。

4. softmax 得到最终后验：

$$p(y = i | x); =; \text{softmax}(L_{\text{comb}})_i$$

5. 用这两通道后验做像素级凸组合：

$$\text{fused} = p_1 \cdot \text{img1} + p_2 \cdot \text{img2}$$

这一步是**贝叶斯风味**：把 CLIP 概率先验转 logits 后与“似然风味的可微证据”相加，最后软决策。

7) 可改进点（实用增强）

1. **自适应门控**：让 λ 随 `confidence` 调整：

$$\lambda(i, j) = \text{clip}(a \cdot \text{conf}(i, j) + b; [0, 1])$$

置信度低的像素，更信可微头；置信度高的像素，更信 CLIP。

2. **多尺度先验**：组合不同 `(patch_size, stride)` 的热力图（如 16/32/48），concat 后再用小头压成先验概率，细节更稳。
3. **提示工程**：
 - 扩充中文/多语种 prompt；
 - 针对你的数据域做 prompt 微调（如“microscopy sharp focus”，“text on sign is readable”等）。
4. **先验后处理**：
 - 对先验概率做轻微 **温度缩放** ($p^{1/\tau}$) 或 **logits 放大/收缩** 调温度；
 - 加 TV 正则/双边滤波平滑先验。
5. **缓存与加速**：
 - 数据集固定时，可把 `focus_map` **离线缓存**（特别是训练阶段），大幅提速。
 - 训练时隔若干 step 更新一次先验（EMA），而不是每个 iteration 都跑 CLIP。

总体来说

先构建可泛化的语义先验（块级→像素级），再和可微统计证据在 logits 空间融合。这既利用了 CLIP 的“开箱即用”语义判别能力，又保持了端到端可训练的灵活性。若接下来需要，我可以帮你把 λ 的**自适应门控**、**多尺度先验**、或 **prompt 优化** 直接写进你现有代码的可插拔版本。

有一些使用clip的论文，但是都主要是红外、可见光融合以及多模态融合而不是多聚焦融合，并且我的使用方法更细化，对整幅图像做 patch 分块、逐块用 CLIP encode，然后拼成 像素级焦点图（soft focus map），而不是只用整体图像语义或图像-文本匹配，并且还有可以修改的点，

我在前面也列出了。